

# JAPANESE PITCH ACCENT TRAINER

## DESIGN DOCUMENT

HAFZA ABDULLAHI  
4<sup>TH</sup> YEAR SOFTWARE DEV STUDENT  
SUBMISSION DATE: 13/02/2026

| CONTENTS                                        |    |
|-------------------------------------------------|----|
| SYSTEM ARCHITECTURE OVERVIEW .....              | 3  |
| Presentation layer (client) .....               | 3  |
| Application Layer (Python + Render) .....       | 3  |
| Render Workflow.....                            | 4  |
| Google SPEECH API VALIDATION .....              | 7  |
| Overview of system .....                        | 9  |
| Data Layer (Firebase + Local JSON Assets) ..... | 10 |
| anki connect api.....                           | 10 |
| USER INTERFACE ARCHITECTURE .....               | 11 |
| Widget Hierarchy .....                          | 11 |
| UI screenshots.....                             | 12 |
| References.....                                 | 13 |
| Software and libraries used.....                | 13 |
| Links .....                                     | 13 |

## SYSTEM ARCHITECTURE OVERVIEW

The Japanese Pitch Accent trainer is built using Flutter and Python. Comprising of both a mobile app and web.

The system is made up of three primary layers

- **Presentation Layer (Client):** A Flutter based application handling the user interaction, audio recording, and visual feedback display.
- **Application Layer (Server):** A Python Flask API hosted on Render, responsible for pitch extraction and graph generation using Praat-based algorithms.
- **Data Layer:** Firebase for authentication and cloud storage, combined with local asset management for Anki deck integration.

## PRESENTATION LAYER (CLIENT)

Responsible for

- User authentication
- Audio recording and playback
- Flashcard rendering
- Graph visualization from backend responses

Flutter compiles to both **Web** and **Android**, a single codebase maintains consistent behaviour across platforms.

## APPLICATION LAYER (PYTHON + RENDER)

This layer performs all digital Signal Processing tasks that would be too heavy for mobile browsers on Render.

**Render[1]** is a modern cloud hosting platform that lets you deploy web apps, APIs, databases, and background workers **without managing your own servers**. The python render server handles:

- Pitch extraction
- Contour smoothing
- Graph generation

Parselmouth (Praat for Python) allows industry standard acoustic analysis without implementing raw DSP manually.

Why is Render used?

This project uses **Parselmouth**, **SciPy**, **Matplotlib**, and other low-level audio libraries. These require:

- system-level dependencies (like libsndfile1)
- Linux audio libraries
- a controlled environment

**Render supports Docker**, meaning you can ship your entire backend (all Python + C++ library requirements) inside a pre-built container.

This guarantees it runs the same way on the cloud as it does on your local machine.

Other platforms (like Firebase Hosting or Vercel) **cannot run Parselmouth** because they don't support such heavy DSP libraries. So, Render was the best option

---

## RENDER WORKFLOW

### 1) User action (client)

- User taps the microphone button or drops/imports an audio file.
- Flutter records audio:
  - Web outputs .webm
  - Mobile outputs AAC/other formats depending on implementation

### 2) Upload to backend (client to Render)

- Flutter sends an HTTP request to:
  - POST /process-audio

### 3) Server receives and stores upload (Render)

- Flask verifies audio exists in request.files.

- The file is written to a temporary file on disk using `tempfile.NamedTemporaryFile(delete=False)` so Parselmouth can access it by path.

#### **4) Convert audio to WAV (FFmpeg via Pydub)**

Pitch extraction and speech recognition require a consistent format.

- The backend attempts:
  - `AudioSegment.from_file(temp_webm)`
  - `audio.export(temp_wav, format="wav")`

If conversion fails (edge case handling):

- If file size is non-zero, the server attempts to use the raw file directly.
- Otherwise, it throws an error

#### **5) Speech transcription (Google Speech Recognition)**

- The server loads WAV using `SpeechRecognition.AudioFile`
- It calls:
  - `recognize_google(..., language="ja-JP")`

#### **Possible outcomes:**

- Success → returns Japanese transcription (kanji/kana depending on recognition)
- `UnknownValueError` → "Could not understand audio"
- `RequestError` → "API Error: ..."

#### **6) Romaji conversion (pykakasi)**

- The transcription is converted to Hepburn romaji:
  - `romaji = "".join([item['hepburn'] for item in result])`

This gives learners a readable fallback even if they can't yet read the kanji.

#### **7) Pitch extraction + graph generation (Parselmouth + Matplotlib)**

- Parselmouth loads the WAV:
  - `snd = parselmouth.Sound(audio_path)`
- Extract pitch contour:
  - `pitch = snd.to_pitch()`

- `times = pitch.xs()`
- `frequencies = pitch.selected_array["frequency"]`

### **Graph output:**

- Matplotlib generates a plot of frequency (Hz) over time (s)
- Title includes combined transcription label:
  - `"{romaji} : {transcription}"`

### **Smoothing:**

- The file includes placeholders for smoothing approaches:
  - moving average, gaussian, savgol
- Currently the server plots the raw contour (and a lightly duplicated plot line); smoothing can be enabled later if desired.

### **8) Response packaging (PNG + headers)**

- The plot is saved to memory (`io.BytesIO()`) as a PNG:
  - `plt.savefig(img_buffer, format="png", dpi=150)`
- Flask sends:
  - PNG bytes as the response body (`send_file(..., mimetype="image/png")`)

### **Transcription is returned via headers:**

- X-Transcription contains URL-encoded Japanese text
- X-Transcription-Romaji contains URL-encoded romaji

URL encoding is necessary because headers may break with raw non-ASCII characters.

### **9) Client receives response and updates UI**

#### **Flutter:**

- Decodes PNG bytes → shows pitch contour image
- Reads headers → displays transcription + romaji
- Updates the flashcard UI to show:
  - pitch graph
  - transcribed text

- (optional) comparison against expected card reading (future feature)

## 10) Cleanup

Server deletes temporary files in finally:

- temp webm
- temp wav

This avoids leaking storage and keeps the container stateless.

---

## GOOGLE SPEECH API VALIDATION

In addition to the graphs, the system uses speech recognition as a first step validation system. It returns what word it heard the user say.

### Educational Benefit

This speech validation layer provides:

- Immediate feedback if the wrong word is spoken
- Reinforcement of vocabulary recall
- Prevention of “graph-only” focus
- Integration of pronunciation + lexical accuracy

Without this layer, a user could produce correct pitch contour on the wrong word.

### Limitations of Speech API Validation

While effective, speech recognition is problematic as it could:

- Misrecognize clear speech
- Return homophones (橋 vs 箸 vs 端) all written as “hashi”
- Fail in noisy environments
- Return kana when flashcard uses kanji

---

## DYNAMIC TIME WARPING

Pitch contours extracted from real speech varies due to:

- speaking speed (fast/slow)
- micro pitch fluctuations
- timing differences between morae (syllables)
- recording latency and trimming differences

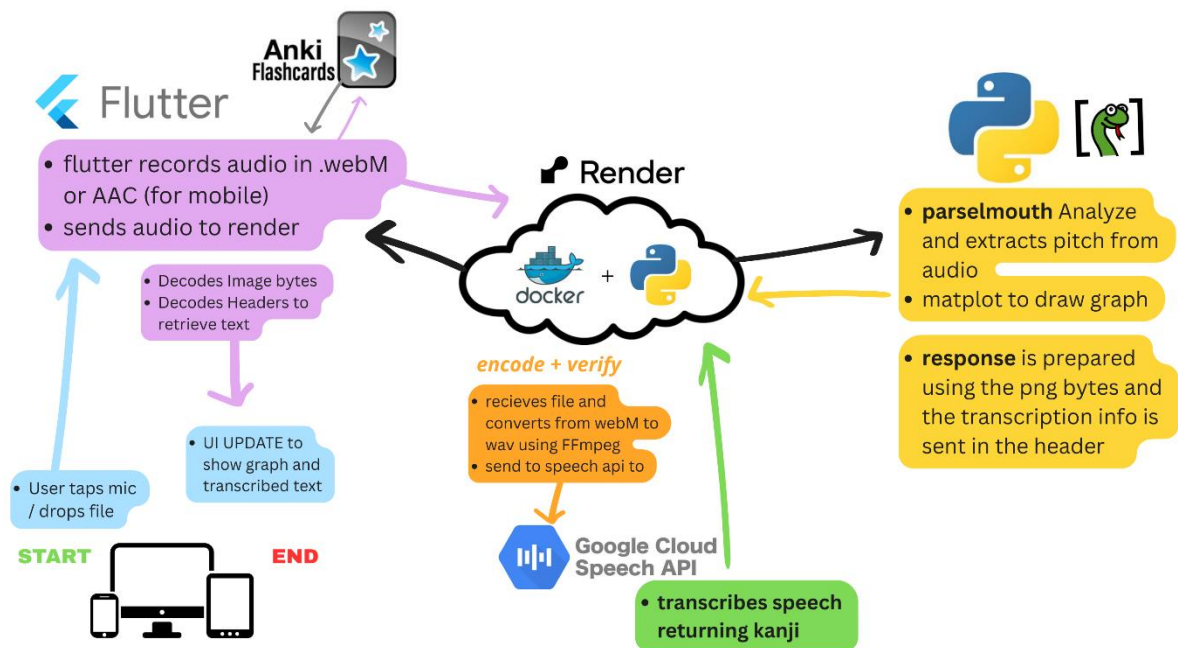
**Dynamic Time Warping (DTW)** solves this by aligning two time series that may differ in length or timing, producing a similarity score based on the best possible alignment

This allows the system to:

- compare a learner's pitch contour to a reference contour properly
- differentiate homophones that differ primarily by pitch pattern
- provide a meaningful score even when timing differs



## OVERVIEW OF SYSTEM



## DATA LAYER (FIREBASE + LOCAL JSON ASSETS)

The project integrates two data sources:

- **Firestore Authentication:** Manages secure user sessions.
- **Local JSON Decks:** Pre-processed Anki flashcards containing vocabulary, readings, example sentences, and audio assets.

## ANKI CONNECT API

To integrate vocabulary from the a deck into flutter, the project uses **AnkiConnect[2]**, a local REST API plugin that exposes the user's Anki data (notes, fields, and media files including audio) through HTTP requests. This retrieves the card info from a premade flashcard deck “***Kaishi 1.5k - Basic Japanese Vocabulary[3]***”

Since Flutter Web and mobile apps **cannot directly read .apkg deck files** (due to SQLite parsing limitations and browser sandboxes), I used preprocessing script in Dart to extract all relevant flashcard information **before packaging the app**, later this will be reworked so the deck is uploaded to the firebase database.

### WIDGET HIERARCHY

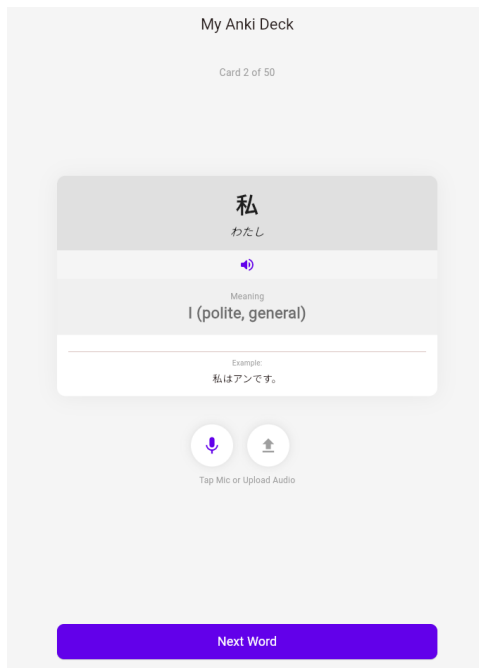
The UI is decomposed into modular, testable widgets, each following the Single Responsibility Principle:

- **WelcomeScreen**  
Determines navigation flow based on authentication state.
- **LoginScreen**  
Implements Google Sign-In using FirebaseAuth, (*currently not implemented*)
- **HomeScreen**  
Displays flashcards, handles recording, transmits audio to the backend, and renders pitch graphs.
  - Displays flashcards
  - Handles recording or file upload
  - Transmits audio to the backend
  - Receives pitch graph + transcription
  - Renders the results inline
- **WordCard Widget**  
Presents:
  - Kanji
  - Furigana
  - English meaning
  - Example sentence
  - Native speaker audio button

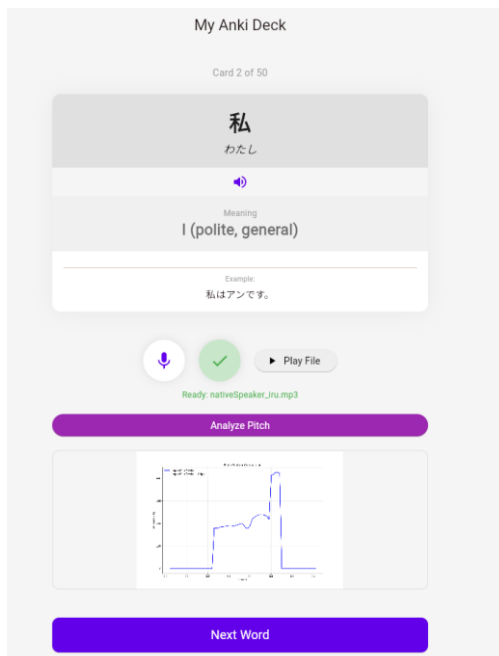
This mirrors standard Japanese learning tools (e.g., Anki), maintaining user familiarity

## UI SCREENSHOTS

App flashcard format as presented to the user before audio has been uploaded



App flashcard after using has uploaded audio from a file and pressed analyse pitch.



## REFERENCES

### SOFTWARE AND LIBRARIES USED

- Flutter SDK (UI framework)
- Flask / Flask-CORS (API + cross-origin support)
- Parselmouth (Praat integration)
- Matplotlib (graphing)
- Pydub + FFmpeg (audio conversion)
- SpeechRecognition (speech-to-text wrapper)
- Google Speech Recognition (via SpeechRecognition backend)
- pykakasi (Japanese → romaji conversion)

### LINKS

- **[1] Render** <https://render.com/>
- **[2] Anki connect**, <https://ankiweb.net/shared/info/2055492159>
- **[3] current anki deck used by project**  
<https://ankiweb.net/shared/info/1196762551>